FRONTEND ARCHITECTURE SPECIFICATION

ZERO-DEPENDENCY VANILLA JS & CSS3

AEGIS-AGENT
Frontend Interface & UX Guide

A deep-dive technical manual detailing the visual design system, dual-theme architecture, interactive dashboard panels, client-side state machine, and real-time backend API synchronization.

Frontend Scope: Pure browser-native HTML5, modern CSS3 with custom properties, and modular Vanilla JavaScript. Operates with zero bundle dependencies (no React, Vue, or Webpack) for 100% offline hackathon reliability and sub-second load times.

INTERFACE PARADIGM

Cybersecurity War Room (Stealth HUD)

VISUAL THEMES

Red & Black Stealth / Warm Ivory Editorial

CLIENT-SIDE ENGINE

Async API Polling • Web Audio Synthesizer

CODE INSPECTION VIEW

Unified & Split Diff Viewer with Gutters

01. Frontend Architectural Philosophy & Tech Stack

The AEGIS-AGENT front end is built strictly on **zero-dependency modern web standards**. Instead of shipping bloated node_modules or requiring complex build steps, the entire UI is delivered via raw HTML5, highly optimized CSS3 custom variables, and a modular JavaScript controller.

Why Zero Dependencies?

- **Instantaneous Cold Starts:** Loads in under 80 milliseconds directly off disk or the local Node.js server.
- **Offline & Sandbox Resilient:** Immune to npm package failures, broken lockfiles, or sandboxed network restrictions during live demonstrations.
- **No Transpilation Friction:** Source code in `index.html`, `css/war-room.css`, and `js/app.js` matches what executes in the browser 1:1.

Frontend File Organization

File Path	Size / Type	Responsibility
`index.html`	~40 KB / Markup	Semantic DOM layout: Persistent banner, navbar, HUD, 2-column war room workbench, dashboard table, and modal dialog.
`css/war-room.css`	~48 KB / Styles	Comprehensive styling: CSS variables, dual-theme palettes, grid/flexbox layouts, custom scrollbars, animations, and diff gutters.
`js/app.js`	~39 KB / Controller	Application state machine: `apiClient` REST integration, DOM caching, 800ms stage polling, tab navigation, and event listeners.
`js/simulator.js`	~13 KB / Web Audio	Client-side Web Audio API synthesizer generating tactile auditory cues for active stage pulses, warnings, and completions.
`js/mockData.js`	~38 KB / Fixtures	Fallback client-side dataset providing deterministic templates for offline testing and initial state seeding.

02. Dual Visual Themes: Stealth vs. Editorial

AEGIS-AGENT features a live 1-click theme switcher in the top navigation bar powered by dynamic CSS variables defined on `:root` and `body.theme-editorial`.

🔴 THEME 1: RED & BLACK STEALTH OPERATIONS (DEFAULT)

Engineered for high-intensity Security Operations Centers (SOC) and war rooms. Uses deep obsidian carbon surfaces paired with glowing crimson ruby accents.

```
--bg-primary: #08090D • --bg-secondary: #0E1017 • --bg-card: #141722
--accent-red: #E63946 • --accent-ruby: #FF2E44 • --text-primary: #F8FAFC
```

🏛️ THEME 2: WARM IVORY EDITORIAL ATELIER

Adapted from high-end minimalist design aesthetics. Uses warm ivory backgrounds, editorial serif headings, muted terracotta accents, and soft taupe borders.

```
--bg-primary: #F4F0E8 • --bg-secondary: #EBE5D8 • --bg-card: #FDFBF7
--accent-terracotta: #A75D43 • --border-taupe: #D8CFC2 • --text-primary: #171512
```

Persistent Theme State

Clicking the theme button in the header toggles `body.classList.toggle('theme-editorial')`, instantly updating every container, border, font color, and diff highlight via CSS variables without reloading the page.

03. War Room HUD & Multi-Agent Workflow Tracker

The War Room view (`#view-warroom`) is the primary interactive cockpit. It is partitioned into a global status HUD, a left-hand stage roster, and a right-hand workbench.



1. Top HUD Components

- **Run Identifier Tag:** Displays current active run (e.g. `RUN-1050`).
- **Laser Progress Track:** Animated crimson bar reflecting synchronous pipeline progress from 0% to 100%.

- **Digital Stopwatch:** High-precision elapsed timer updated synchronously from the server's execution timestamp.
- **Pipeline Network Map:** Visual network flow highlighting active nodes (System, Red, Sentinel, Fix, Auditor).

2. Dynamic 5-Stage Live Timeline

Populated dynamically by `renderTimelineFromApi(run)`. Each stage displays an indicator (✓, ●, ⚠️, ✕), the agent role tag, completion timestamp, and activity description.

04. The Four Workbench Detail Panes

The right-hand column houses the **Investigation and Remediation Workbench**, split across four dedicated tab panels accessible via click or keyboard shortcuts (1–4).

Pane 1: Finding & Evidence View (1)

- **Severity Pill & CVSS:** Color-coded severity badge (CRITICAL, HIGH, MEDIUM) and CVSS score.
- **Component & CWE:** Target URL/parameter and official CWE identifier (e.g. CWE-89, CWE-287, CWE-22, CWE-78).
- **Interactive Evidence Viewer:** Sub-tabbed code viewer displaying:
  1. `HTTP Request`: Raw probe headers, query parameters, and test payload.
  2. `HTTP Response`: Server response headers and reflected syntax error / leaked content.
  3. `Raw Trace`: Database query failure or signature match logs.
  4. `Call Stack`: Application call stack identifying the vulnerable source code line.

Pane 2: Proposed Fix (Diff) View (2)

- **Technical Rationale:** Clear explanatory card explaining *why* the defensive mitigation works.
- **Allowlisted Test Validation Badge:** Displays the syntax validation command executed (e.g. `python -m py_compile`) with a green `PASSED` pill.
- **Interactive Code Diff Viewer:**
  - **Unified Mode:** Displays inline deletions (red) and additions (green) with line gutters and comment callouts.
  - **Split Mode:** Side-by-side comparison of original vs. remediated code.
  - **Diff Stats Pill:** Real-time addition/deletion counters (e.g. `+6 / -4`).
- **Human Approval Gate:** Two action buttons (**Approve for Demo** / **Reject Fix**). Clicking either sends an API request to the backend and updates the status badge without touching repository files.

Pane 3: Regression Results View (3)

- **Reproducibility Status Banner:** Prominent banner displaying either:
  - `🛡️ REMEDIATED — Not Reproducible` (Emerald): All 4 checks passed.
  - `⚠️ REPRODUCIBLE — Secondary Bypass Detected` (Crimson): Vulnerability persists.
- **4-Check Matrix Grid:** Detailed breakdown of each independent regression test, including test ID, expected status, actual status, and description.
- **Interactive Controls:** Includes a **Live Replay Suite** button and a **Toggle Outcome** test button.

Pane 4: Live Telemetry Logs (4)

Real-time streaming terminal view showing inter-agent telemetry messages, timestamps, and coordinator state transitions. Automatically scrolls to bottom on new events.

05. Dashboard Table, Modal Dialog & State Engine

1. Dashboard & History View (`#view-dashboard`)

Accessible via the top navigation bar. Provides a high-level operational overview:

- **Metric Cards:** Total local scans, high severity findings, remediations, and average TTR.
- **Search & Filter Bar:** Real-time client-side text filter matching Run IDs, target names, or CWE titles, alongside filter chips (`All` , `Complete` , `Needs Review` , `Failed`).
- **Historical Runs Table:** Lists seeded and newly created runs with status pills, timestamps, and **Inspect Run** buttons that load that run directly into the War Room.
- **Export Audit Report Button:** One-click download of the complete sanitized JSON report (`AEGIS-Audit-Report-{runId}.json`).

2. "Configure New Demo Run" Modal Dialog (`#modal-new-run`)

An engineered modal dialog (N) designed with responsive constraints and safety bounds:

- **Fixed Header & Footer:** Header and action buttons (Cancel / Start Run) remain permanently visible and clickable; the middle form body scrolls smoothly with a custom dark-cyber scrollbar.
- **Locked Target Environment:** Hardcoded to authorized local container fixtures with a `LOCKED` badge to prevent arbitrary external targeting.
- **Test Case Selector (4 Cards):** Selectable option cards with custom hover glows for CWE-89, CWE-287, CWE-22, and CWE-78.
- **3-Column Simulation Outcome Cards:**

  ● Complete (Pass) ▲ Needs Review ✕ Regression Fail

- **Mandatory Scope Checkbox:** Disables the "Start Run" button with 50% opacity until the user explicitly checks the scope authorization consent.

3. Client-Side State Management in `js/app.js`

```
// Central Application State Object
const state = {
  activeNav: 'war-room',          // 'war-room' | 'dashboard' | 'logs' | 'about'
  activeDetailTab: 'finding',     // 'finding' | 'fix' | 'regression' | 'rawlogs'
  activeEvidenceTab: 'request',   // 'request' | 'response' | 'hex' | 'stack'
  diffViewMode: 'unified',        // 'unified' | 'split'
  dashboardFilter: 'all',         // 'all' | 'complete' | 'needs_review' | 'failed'
  searchQuery: '',
  selectedTestCaseId: 'input-validation',
  currentRunData: null,           // Live run object returned from backend
  pollTimer: null,                // Polling timeout reference (800ms intervals)
  recentRuns: []                  // Cached historical runs list
};
```

4. Web Audio Synthesizer in `js/simulator.js`

Uses browser-native `window.AudioContext` to generate custom tactile auditory frequencies:

- `stage_advance` : 440 Hz → 880 Hz upward frequency chirp.

- `alert`: 320 Hz dual-pulse square wave for Sentinel warnings.
- `run_done`: Harmonious 3-tone chime (523 Hz → 659 Hz → 784 Hz).